

Distributed Training & Orchestration

Parallelism · Kubernetes · SLURM

Agenda

- Framing the Scaling Challenge
 - Parallelism Strategies
 - DDP in Practice
 - Kubernetes for GPU Workloads
 - Torchrn + k8s job
 - Distributed Training with SkyPilot
- Slurm on Kubernetes via Soperator
 - Reliability & Performance
 - Infrastructure as Code
 - Training Monitoring
 - Troubleshooting & Common Pitfalls

Learning Objectives

- Explain why distributed training is necessary
- Recall parallelism strategies
- Implement PyTorch DDP
- Deploy GPU workloads on Kubernetes
- Run multi-node jobs with Kubernetes and Slurm-on-Kubernetes (Soperator)
- Provision GPU clusters with Terraform
- Monitor training health

Framing the Scaling Challenge

Why We Need Distributed Training

Model-Size Problem

Modern LLMs (7B–700B+ params) exceed single-GPU HBM.
B200 = 192 GB
B300 = 288 GB — still not enough.

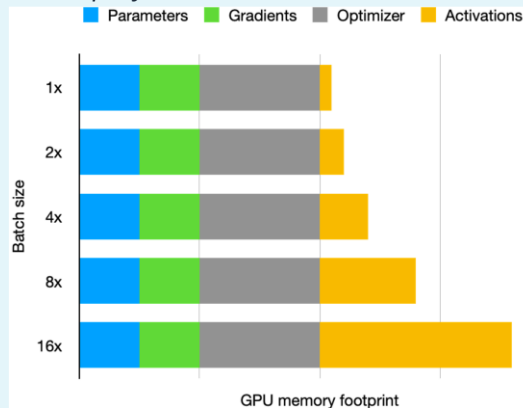
Speed Problem

Training on trillion-token datasets on one GPU = months to years.
Only parallelism across hundreds of GPUs is practical.

Memory-Wall Problem

Adam stores $\sim 2\times$ params in optimizer
Activations scale with sequence length

example for FP32



Memory Breakdown for Training

Model	Params	Weights (BF16)	Gradients (BF16)	FP32 Master Weights	Adam States	Model State Total	Activation Memory (typical)	Total Training Memory (rough)
Llama 3 8B	8B	16 GB	16 GB	32 GB	64 GB	128 GB	~40–160 GB	~168–288 GB
Llama 3 70B	70B	140 GB	140 GB	280 GB	560 GB	1.12 TB	~300 GB–1.4 TB	~1.4–2.5 TB
Llama 3 405B	405B	810 GB	810 GB	1.62 TB	3.24 TB	6.48 TB	~1.5–8 TB	~8–14.5 TB

How We Measure Training Efficiency

- **Training Metrics**

- **Time-to-train**
wall-clock from start to convergence
- **MFU (Model FLOPs Utilisation)**
 $\text{observed FLOP/s} \div \text{peak FLOP/s}$ (target >40%)
- **Tokens/sec or samples/sec**
raw throughput
- **GPU utilisation:** target >80%
- **MTTR:** mean time to recovery from failures
- **Cost-per-epoch (\$):** total training cost

- **Inference Metrics (contrast)**

- Latency
- Time-to-first-token (TTFT)
- Requests per second
- Availability / uptime

Four Key Scaling Bottlenecks

Gang Scheduling

All GPUs must start simultaneously. One missing node delays the entire job.

Network Bottleneck

Inter-node IB bandwidth is 10–100× narrower than intra-node NVLink. Comm dominates if not overlapped with compute.

Stragglers

One slow GPU (thermal, memory error, flaky NIC) forces all others to wait at sync barriers.

Pipeline Bubbles

Idle GPU time during ramp-up/drain can reach 20–30% without micro-batch tuning.

Parallelism Strategies: How to Split the Work

Parallelism Strategies

- **Data Parallelism (DDP)** - Each GPU holds a full model copy, processes a different data shard
- **ZeRO-3 / FSDP2** - Full parameters sharding
- **Tensor Parallelism (TP)** - Split individual weight matrices across GPUs
- **Pipeline Parallelism (PP)** - Model layers grouped into stages, each on a different GPU/node
- **Sequence Parallelism (SP)** - shards LayerNorm/Dropout along sequence dim across same TP GPUs
- **Context Parallelism (CP)** - splits long input sequences across GPUs for >32K token training
- **Expert Parallelism (EP)** - Each GPU holds a subset of experts

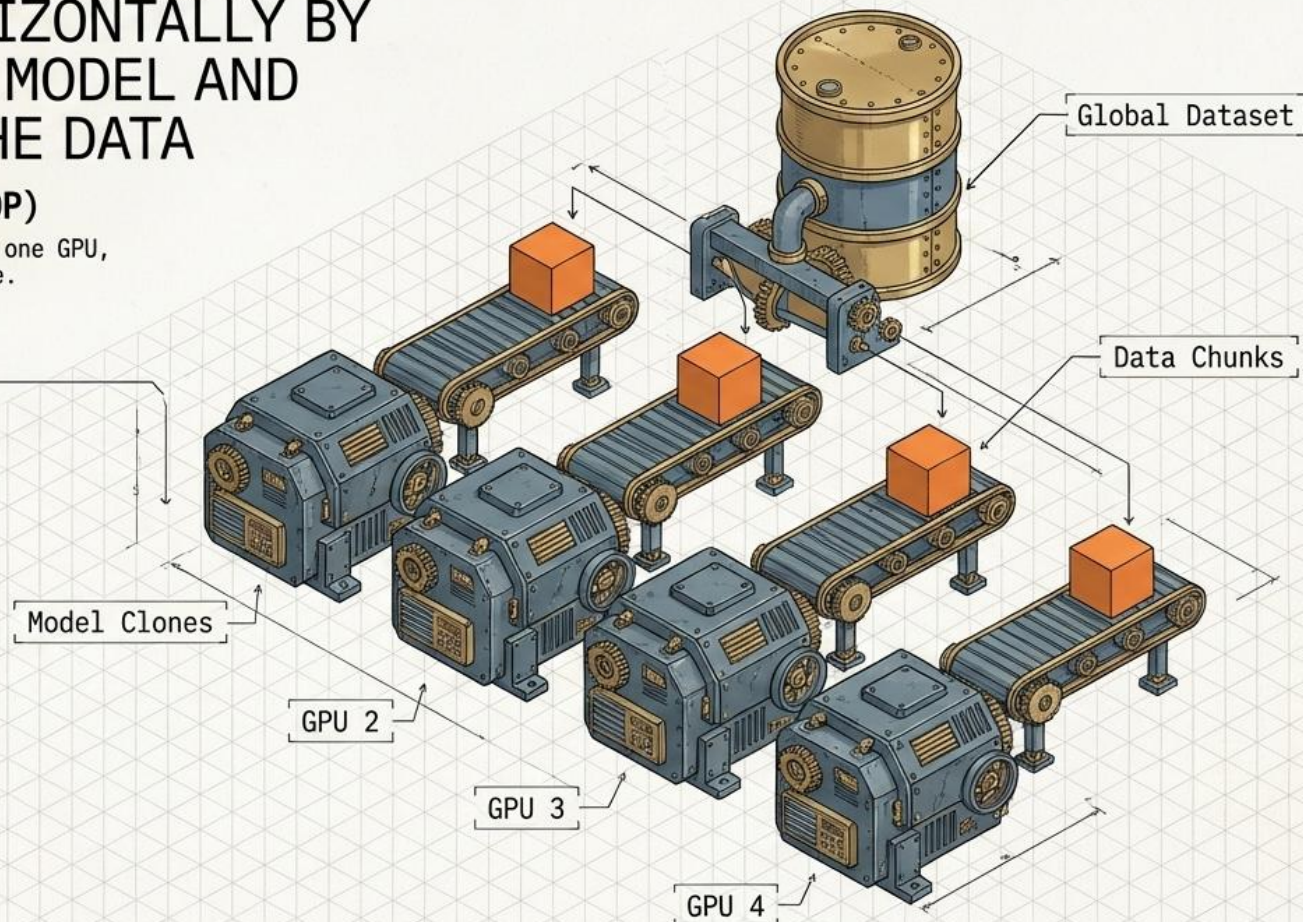
SCALING HORIZONTALLY BY CLONING THE MODEL AND SPLITTING THE DATA

Data Parallelism (DP)

The model fits perfectly on one GPU,
but the dataset is too large.

The Mechanism: Each GPU
receives an exact clone of
the model but processes a
different subset of the
data simultaneously.

Distributed Samplers:
Ensures data is divided
evenly, each sample is
seen exactly once per
epoch, and the shuffling
seed is synchronized
globally.



Quick Reference: Parallelism Strategy Selection

Scenario	Strategy	Config	Tool
Model fits 1 GPU, faster training	Data Parallelism (DDP)	N GPUs, same model	PyTorch DDP / torchrun
Barely fits 1 GPU (7B–13B) params replicated, states/grads sharded	ZeRO-2 Sharded DP	ZeRO-2 + DDP, 8–64 GPUs	DeepSpeed
Doesn't fit 1 GPU (13B–70B) pure PyTorch, full param sharding	FSDP2 / ZeRO-3	FSDP2 / ZeRO-3, 8–128 GPUs	PyTorch FSDP2
Doesn't fit, Megatron stack (30B+)	Tensor Parallel + ZeRO-3	TP=8 within node	Megatron-LM / NeMo
Very large (70B+) / fastest training	4D: DP + TP + PP + SP	TP=8, PP=4–16, DP=N	Megatron-LM / NeMo
Long context (>32K tokens)	Add Context Parallelism	+CP=4–8 on top of 4D	Megatron-LM / ring-attn
Sparse MoE (DeepSeek, Mixtral)	Expert Parallelism + 4D	EP all-to-all + TP within node	Megatron-Core MoE

Hybrid Parallelism: 4D / 5D Configurations

Bandwidth topology drives placement decisions:

TP (Tensor)	Within node	NVLink ~900 GB/s	2–8 GPUs
PP (Pipeline)	Across nodes in rack	InfiniBand 400 Gb/s	Point-to-point
CP (Context)	Across nodes	InfiniBand	>32K sequences
DP / FSDP	Broadest scope	InfiniBand + topology	All replicas
EP (Expert)	All EP-group GPUs	All-to-all IB	MoE only

Llama 3 405B: TP=8, PP=16, CP=4, DP=32 $\rightarrow 8 \times 16 \times 4 \times 32 = 16,384$ H100 GPUs

Memory-Efficiency Techniques

FlashAttention-2/3

IO-aware SRAM-tiled attention.
Reduces attention memory $O(n^2) \rightarrow O(n)$.
Default in Megatron, HF Transformers, NeMo.
FA-3 adds FP8 support for H100.

Comm-Compute Overlap

AllReduce / AllGather overlapped
with backward computation.
NCCL algorithm + chunk size tuning
needed to achieve overlap at scale.

Gradient Checkpointing

Save only subset of activations.
Recompute discarded ones during backward.
Reduces activation memory ~70%
at cost of ~30% extra compute.

Gradient Compression

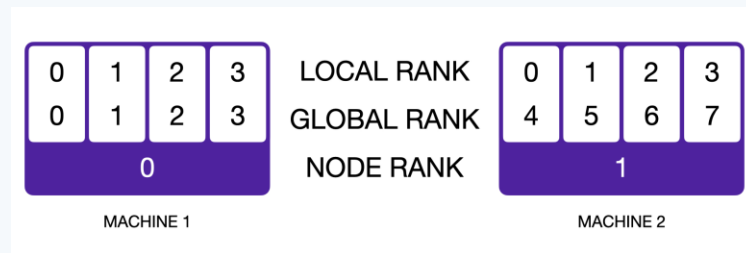
Top-k sparsity: 90%+ bandwidth reduction.
Lossy quantization: lower precision in comms.
Error compensation prevents quality drift
over many training iterations.

Distributed Training in Practice

Process Groups & Multi-Node Initialization

```
def train(local_rank, local_world_size, node_rank, num_nodes, args):  
    global_world_size = local_world_size * num_nodes  
    global_rank = (node_rank * local_world_size) + local_rank  
  
    # Initialize NCCL process group  
    dist.init_process_group(backend='nccl')  
    # global_rank=0 on node 0 GPU 0, global_rank=1 on node 0 GPU 1, etc.  
  
    print(f"World size: {dist.get_world_size()}, Rank: {dist.get_rank()}")
```

- **$\text{global_rank} = \text{node_rank} \times \text{GPUs_per_node} + \text{local_rank}$**
- NCCL backend: required for GPU-to-GPU communication (uses NVLink + IB)
- Timeout: add `init_method timeout` to avoid silent hangs on large clusters



DDP Training Loop & DistributedSampler

```
ddp_model = DDP(model, device_ids=[rank % torch.cuda.device_count()])

train_sampler = DistributedSampler(train_dataset, num_replicas=world_size, rank=rank)
train_loader = DataLoader(train_dataset, batch_size=args.batch_size, sampler=train_sampler)

for epoch in range(args.epochs):
    train_sampler.set_epoch(epoch)  # ← REQUIRED for correct per-epoch shuffling
    for inputs, targets in train_loader:
        optimizer.zero_grad()
        outputs = ddp_model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()  # ← DDP triggers AllReduce here automatically
        optimizer.step()
```

- `loss.backward()` triggers: (1) local grad compute → (2) NCCL AllReduce → (3) identical param update on all GPUs
- `DistributedSampler`: divides dataset evenly, shuffles differently per epoch using epoch as seed
- Forgetting `set_epoch()` → all GPUs see identical data order every epoch → training quality degrades

Checkpointing in Distributed Settings

```
def save_checkpoint(model, optimizer, epoch, args, is_best=False):
    if dist.get_rank() != 0:  # Only rank 0 saves — avoids write conflicts
        return
    checkpoint = {
        'epoch': epoch,
        'model_state_dict': model.module.state_dict(), # .module unwraps DDP
        'optimizer_state_dict': optimizer.state_dict(),
        'args': vars(args)
    }
    torch.save(checkpoint, f"{args.checkpoint_dir}/checkpoint_{epoch}.pt")

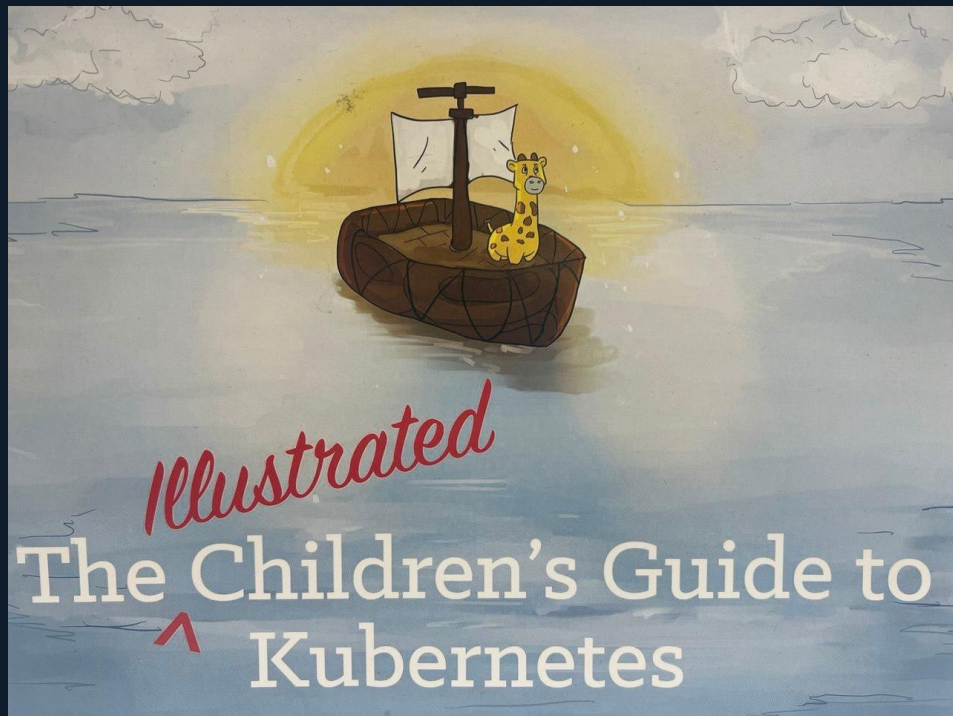
# On resume — broadcast from rank 0 to keep all processes in sync:
epoch_tensor = torch.tensor([start_epoch], device=device)
dist.broadcast(epoch_tensor, src=0)
start_epoch = int(epoch_tensor.item())
```

- Only rank 0 writes — prevents file conflicts on shared storage
- `model.module.state_dict()` — the `.module` unwraps the DDP wrapper
- Broadcast `start_epoch` after load — ensures all ranks agree on resumption point
- ZeRO-3/FSDP2: use PyTorch DCP (Distributed Checkpoint) for sharded save/load



Break — 10 minutes

Kubernetes



Illustrated
The Children's Guide to
Kubernetes

What is Kubernetes

- **Cluster management system**

- Open sourced by Google
- Support high availability clusters
- Highly scalable

POD

- Usually single container for ML load
- Packed with dependencies like CUDA

Service

- Has IP address
- Abstracts POD access

Operators

- Simplify management of complex applications
- Simply a controller that watches events of custom resource and take actions

Storage

- In container storage nonpersistent
- Persistent Volumes supported for pods

Why Kubernetes

Building and deploying AI systems at scale means juggling complex infrastructure — and that's where Kubernetes shines.

From managing GPU resources to scaling inference endpoints,

Kubernetes brings structure and automation to the chaos of machine learning pipelines.

- ✓ Simplifies model deployment and pipeline management
- ✓ Automates scaling from experimentation to production
- ✓ Increases resilience with self-healing infrastructure

How Kubernetes handles AI workloads

- **Model training**

- Jobs run to completion — auto-restarted on failure
- Distributed training with PyTorch
- Each process in a separate pod, orchestrated by K8s
- Reusable pipeline components across teams & clusters

- **Model Inference**

- Deployments maintain required model replicas
- Auto-scales with traffic; restores failed replicas
- Multiple model versions
- A/B testing for experimental model comparisons

K8s key benefits for AI teams



Elastic Scaling

Spin up dozens of processes for training, release resources when idle. Demand-based scaling.



Fault Tolerance

Auto-restarts failed pods, zero-downtime rollouts, instant rollback. Less manual recovery.



Resource Isolation

Namespaces, quotas & priority classes prevent GPU/CPU contention across teams.



Reproducibility

Config-as-code: pipelines run identically on any cluster, in any environment.



CI/CD Integration

Automated builds, model validation, staged deploys. GitOps with ArgoCD or Helm.



Full Observability

Prometheus + Grafana for metrics, Loki/ELK for logs, GPU monitoring built in.

Kubernetes (MK8s) Fundamentals for GPU Workloads

Prerequisites for GPU workloads on K8S

- **On the Node (worker node level)**

First, the physical/OS layer:

- **NVIDIA drivers** installed on the host OS
- **CUDA toolkit** (for NVIDIA) — needed by most ML workloads
- **Container runtime with GPU support** — either containerd with the NVIDIA container toolkit, or nvidia-docker2
- **NVIDIA Container Toolkit** — this is the bridge between the container runtime and the GPU

On the Cluster (Kubernetes level)

NVIDIA GPU Operator — the recommended all-in-one solution from NVIDIA. It automatically manages driver installation, the device plugin, monitoring, and more via a single Helm chart. Covers:

- NVIDIA Device Plugin
- DCGM Exporter (metrics)
- Node Feature Discovery (NFD)
- GPU Feature Discovery (GFD)
- MIG Manager (if using multi-instance GPU)

OR manually install just the **NVIDIA Device Plugin** (k8s-device-plugin) — simpler but you handle drivers yourself. This is what exposes nvidia.com/gpu as a schedulable resource.

Node Feature Discovery (NFD) — labels nodes so the scheduler knows which nodes have GPUs.

Kubernetes Resource Primitives for GPUs

- GPU capacity exposed via NVIDIA device plugin as **nvidia.com/gpu** resource
- Node pools grouped by GPU type with node labels for affinity
- Taints on GPU nodes: prevent CPU-only workloads from consuming expensive GPU slots
- Training pods use matching tolerations to land on GPU nodes
- Resource requests **MUST** equal limits for GPUs — Kubernetes enforces request == limit
- Setting requests without limits leaves GPUs unaccounted in scheduling decisions

Then in your pod spec, request GPUs like:

```
resources:
  requests:
    nvidia.com/gpu: "8"    # Must equal limits for GPU resources
  limits:
    nvidia.com/gpu: "8"
```

GPU Driver Deployment: Two Paths on Nebius MK8s

- **Path 1: Driverfull Boot Images (default)**

- Pre-built images with GPU drivers, CUDA toolkit, nvidia-container-toolkit, device plugin, and IB drivers baked in
- Preset controls driver series and CUDA version
- No operator installation needed — cluster is GPU-ready immediately
- **Recommended for most Nebius customers**

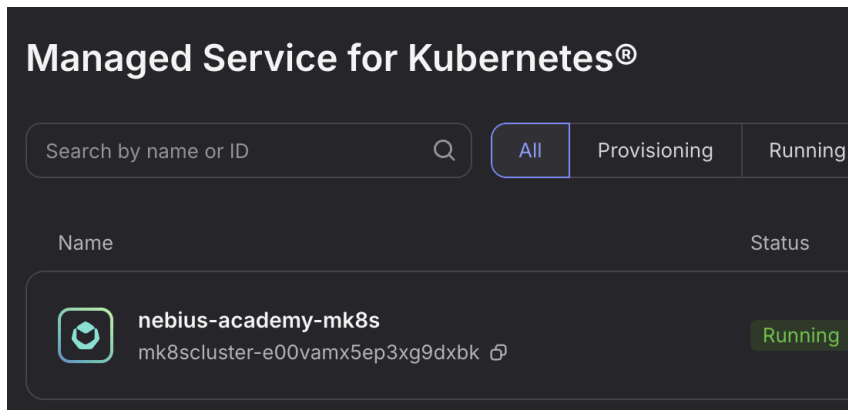
- **Path 2: Standard Images + Operators**

- Use when: custom driver version
- NVIDIA GPU Operator required for any GPU node not using driverfull image
- NVIDIA Network Operator required for InfiniBand cluster
- Operators take care of all drivers on every node
- **On generic K8s: operators always needed from scratch**

Managed Service for Kubernetes on Nebius

With Kubernetes, you can efficiently **manage** and **scale** your containerized ML/AI applications and ensure they are **portable and fault-tolerant**.

Nebius AI Cloud offers Managed Service for Kubernetes, which simplifies cluster deployment and management for Kubernetes.



DEMO: create MK8S on Nebius platform

Gang Scheduling: Volcano vs Kueue

- **Volcano**

- HPC-style gang scheduler for Kubernetes
- all-or-nothing GPU allocation
- Prevents partial scheduling deadlocks
- **Best for: pure HPC training clusters**

- **Kueue**

- Works with default K8s scheduler — doesn't replace it
- Cluster-wide quotas
- **Best for: mixed training + inference, multi-team clusters**

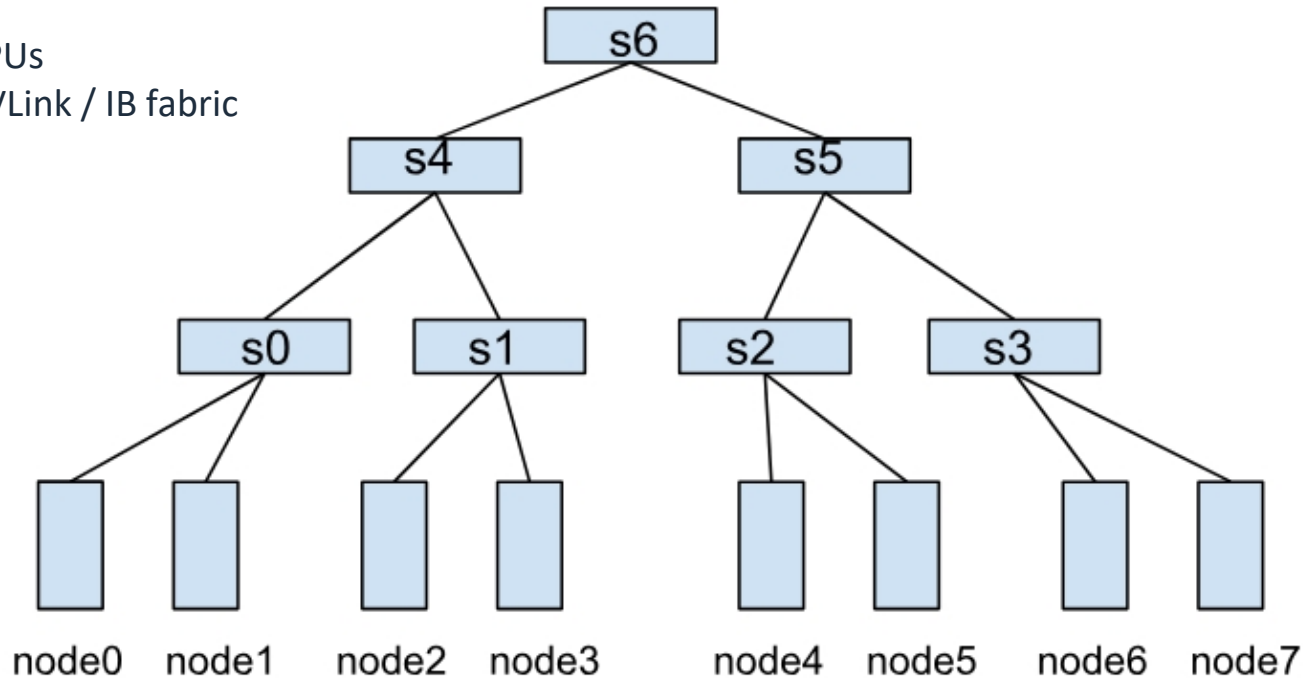
Why it matters: partial scheduling = deadlock. Running pods hold quota; pending pods wait forever.
Gang scheduling ensures all N GPUs start simultaneously or none do.

NVIDIA Data Center GPU Manager (DCGM)

- **DCGM — Cluster-Level GPU Health**
 - Run on every GPU node, 50+ metrics per GPU
 - GPU utilisation, memory BW, NVLink throughput, ECC errors (detected by Error-Correcting Code), XID events
- **Critical XIDs — drain immediately:**
 - XID 74: NVLink error
 - XID 79: GPU reset required
 - XID 92–94: ECC uncorrectable
- <https://docs.nvidia.com/deploy/xid-errors/analyzing-xid-catalog.html>

Topology aware scheduling

- Places pods on GPUs
nodes close on NVLink / IB fabric



Simple setup with Torchrun



How it works conceptually

Multinode training involves deploying a training job across several machines.

There are few ways to do this:

- running a `torchrun` command on each machine with identical rendezvous arguments,
- or deploying it on a compute cluster using a workload manager (k8s/slurm)

torchrun

PyTorch offers a utility called `torchrun` that provides multinode training functionality.

`torchrun` needs each process to know:

- `MASTER_ADDR` — hostname of the rendezvous
- `MASTER_PORT` — port for rendezvous
- `WORLD_SIZE` — total number of processes (nodes × GPUs per node)
- `NODE_RANK` — which node this pod is (0, 1, 2...)

Master address and port

With `master_addr:master_port`, **rank 0 explicitly creates a TCPStore server** on that `address+port`. All other ranks connect to it as clients.

The TCPStore is used for two things:

- **Rendezvous** — all ranks check in, agree on world size, get their rank assignments
- **Key-value barrier** — coordinate during init (until all ranks have joined)

Once all ranks have connected and `init_process_group` completes, **NCCL takes over** for the actual gradient communication — `MASTER_ADDR/PORT` is no longer involved.

Rule of thumb: `MASTER_ADDR` must always be the IP of the **rank 0 machine**, and that port must be open and reachable from all other nodes.

Training script torchrun initialization

Your script should use **torchrun**-compatible initialization:

```
import torch
import torch.distributed as dist

def main():
    dist.init_process_group(backend="nccl") # or "gloo" for CPU

    local_rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)

    # ... your training loop

    dist.destroy_process_group()
```

torchrun automatically sets LOCAL_RANK, RANK, WORLD_SIZE — you don't set those manually in code.

Manual Multinode DDP demo with torchrun

1. **Create 2 VMs with single GPU each. SSH to VMs**
2. **Update package list:** `sudo apt update`
3. **Install venv:** `sudo apt install python3-venv -y`
4. **Create environment:** `python3 -m venv pytorch_env`
5. **Activate environment:** `source pytorch_env/bin/activate`
6. **Install PyTorch:**
`pip3 install numpy torch --index-url https://download.pytorch.org/whl/cu130`
7. **Check:** `torchrun --version`
8. **Clone test repo:** `git clone https://github.com/pytorch/examples.git`

Start torchrun manually on both nodes

```
torchrun \  
--nnodes 2 \  
--nproc_per_node 1 \  
--node_rank 0 \  
--master_addr 10.6.35.50 \  
--master_port 29500 \  
/mnt/data/examples/distributed/ddp-tutorial-series/multinode.py 50 10
```

```
torchrun \  
--nnodes 2 \  
--nproc_per_node 1 \  
--node_rank 1 \  
--master_addr 10.6.35.50 \  
--master_port 29500 \  
/mnt/data/examples/distributed/ddp-tutorial-series/multinode.py 50 10
```

Common gotchas

- **Pods start at different times** — torchrun has a built-in timeout for rendezvous, but if your image pull is slow, increase `--rdzv_timeout`.
- **NCCL between nodes needs open ports** — make sure your NetworkPolicy (if any) allows traffic between pods on arbitrary ports, not just 29500.
- **GPU affinity** — set `--nproc_per_node` to match exactly the number of GPUs on node
- **To debug logs:**
`export NCCL_DEBUG=INFO`

Distributed Training with SkyPilot

SkyPilot: Abstracting 'Where' from 'What'

- Open-source framework:
same YAML runs on Nebius MK8s, AWS EKS, GCP GKE — no training code changes
- Automatic NCCL env vars, node IPs, GPU count — no cluster-specific boilerplate in training code
- Container-first: training code lives in a Docker image, fully portable

```
# sky_multi_node_ddp.yaml
resources:
  cloud: nebius
  accelerators: H100:8
  region: eu-north1
num_nodes: 2
setup: |
  uv pip install torch>=2.6.0 scikit-learn
run: |
  MASTER_ADDR=$(echo "$SKYPILOT_NODE_IPS" | head -n1)
  torchrun --nnodes=$SKYPILOT_NUM_NODES \
    --node_rank=$SKYPILOT_NODE_RANK \
    --nproc_per_node=$SKYPILOT_NUM_GPUS_PER_NODE \
    --master_addr=$MASTER_ADDR --master_port=12355 \
    train.py
```

Managed SkyPilot API Server on Nebius

Fully managed, one-click solution designed to automate the deployment and operation of the SkyPilot control plane.

One-Click Deployment: Provision a dedicated API server directly from the Nebius Console

Team Collaboration: Enables multiple users to share a single endpoint

High Availability: Features a fault-tolerant architecture backed by Nebius

Unified Visibility: Provides a "single pane of glass" view for all running jobs

Asynchronous Execution: Users can launch multiple parallel experiment batches

Slurm on Kubernetes via Soperator

Soperator Architecture

- **What is Soperator?**

- World's first production Slurm-on-K8s Operator (Nebius open source)
- Slurm control plane runs as Kubernetes Pods
- K8s handles pod lifecycle; Slurm handles job queuing + accounting

- **Pod roles:**

- Login pods: SSH access, sbatch / srun commands
- Controller pod: slurmctld — K8s restarts in seconds if it crashes
- Worker pods: one per node, mapped to K8s GPU node

- **The 'Jail' — Shared Root Filesystem**

- All Slurm nodes share identical root FS via PV mounted at /
- Install package on login node → available everywhere immediately
- Download dataset once → all workers see it
- Indistinguishable from bare-metal Slurm UX

- **High Availability (HA) Story:**

- slurmctld crash → K8s respawns in seconds → state restored from Slurmdbd PVC
- vs bare-metal: hours of manual intervention

Slurm Configuration: Multi-Node DDP Example

```
# sbatch.sh
#!/bin/bash
#SBATCH --job-name=multi-node-ddp
#SBATCH --nodes=2
#SBATCH --gres=gpu:8
#SBATCH --cpus-per-task=120

cd $SLURM_SUBMIT_DIR
source venv/bin/activate
srun srun.sh

# srun.sh
export GPUS_PER_NODE=8
HOST_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)

torchrun --nnodes $SLURM_NNODES \
  --nproc_per_node $GPUS_PER_NODE \
  --master_addr $HOST_ADDR --master_port 12355 \
  --node_rank=$SLURM_NODEID \
```

Same train.py — only the launcher config differs. Slurm provides SLURM_NNODES, SLURM_NODEID; SkyPilot provides SKYPILOT_NUM_NODES, SKYPILOT_NODE_RANK.

Reliability & Performance Optimisation

Automated Health Checks at Scale

At cluster scale, failures are statistically certain — not exceptional.

ByteDance Key Deployment Results (3-Month Training Job on 9,600 GPUs):

Total Explicit Failures: 38,236 (detectable errors like CUDA errors, CPU OOM, or IB errors).

Total Implicit Failures: 5,948 (undetectable "ghost" failures, such as job hangs or performance degradation).

- **Slurm HealthCheck: script runs on each node every 60s**
 - nvidia-smi: verify all GPUs visible and not in error state
 - DCGM ECC counters: single-bit = monitor; double-bit = drain immediately
 - NCCL loopback test: verify intra-node GPU-to-GPU communication
 - InfiniBand status check: catch flaky network interfaces
- **NCCL Bandwidth Benchmarks: run all_reduce_perf periodically across the full cluster**
 - Nodes >20% below cluster median → auto-drain (prevents straggler effect)

Checkpointing Strategy: RPO, RTO & Async

- **Recovery Planning**

- RPO (Recovery Point Objective): max acceptable progress loss
- Typical RPO = 30–60 min → checkpoint every 500–1,000 steps
- RTO (Recovery Time Objective): time from failure to resumed training
- Dominated by checkpoint load time + job re-launch latency

- **Sharded Checkpointing:**

- Each rank saves its shard independently — no AllGather cost
- Requires same topology at resume time

- **Asynchronous Checkpointing**

- **Traditional:** all GPUs pause while writing to storage (can be minutes)
- **Async:** copy state to CPU RAM (fast), then stream to object storage in background
- **Flash Checkpoint:** latest checkpoint stays in CPU RAM → recovery in seconds

- **Cost of not checkpointing:**

- A 70B model on 512 GPUs = ~\$50K/hr.
4 hours of wasted compute on failure = \$200K lost

Elastic Training

- **Elastic Training**
 - Add/remove nodes mid-run without full restart
 - PyTorch Elastic (torchelastic / torchrun) supports dynamic world-size
 - Use cases: graceful spot preemption, absorbing new capacity
 - Requires training loop to handle membership changes at sync points
 - Not all model-parallel configs support this



Break — 10 minutes

Infrastructure as Code



What is Terraform and IaC

Terraform is an **infrastructure as code (IaC)** tool made by HashiCorp that lets you define, provision, and manage cloud infrastructure using configuration files instead of manual point-and-click in a console.

The core idea: You write a config file describing what infrastructure you want and Terraform figures out how to create or update it to match that desired state.

How it works:

You write **.tf** files declaring resources — e.g. "I want an VM instance of type H100"

You run **terraform plan** — it shows you what it *will* do without making changes yet

You run **terraform apply** — it provisions the actual infrastructure

Terraform tracks what it created in a **state file**, so future runs only change what's different

Why people use it:

Changes are reviewable and version-controlled, just like code

Destroying or recreating environments is repeatable and predictable

Teams can collaborate on infrastructure the same way they collaborate on code

Terraform for GPU Cluster Provisioning

- **Key Terraform resources for Nebius GPU clusters:**
 - `nebius_mk8s_cluster` — managed Kubernetes control plane
 - `nebius_mk8s_node_group` — GPU node pool: instance type, disk, autoscaling min/max, labels/taints
 - `nebius_vpc_network` / `nebius_vpc_subnet` — cluster networking
 - `nebius_storage_filesystem` — shared storage for datasets and checkpoints
- **State management:**
remote backend (Nebius Object Storage, S3-compatible)

Nebius terraform solution library

- TF Solutions library
<https://github.com/nebius/nebius-solutions-library>
- ML cookbook
<https://github.com/nebius/ml-cookbook>

Training Monitoring



Healthy metrics

Metric	Healthy Range	Action if Degraded
GPU Utilisation	> 80%	Profile for CPU bottlenecks, data loading, excess sync
Memory BW Utilisation	> 70%	Check batch size / sequence length
NVLink Throughput	Near peak ~900 GB/s	Investigate TP communication patterns
ECC Single-bit Errors	0 or very low	Monitor trend; drain if increasing
XID Errors	0	Drain immediately: XID 74, 79, 92–94

Training-Level Metrics & NCCL Debugging

- **Training-Level KPIs**

- Throughput: tokens/sec — primary KPI. 10% drop = investigate immediately
- MFU = $(\text{tokens/sec} \times \text{FLOPs/token}) \div (\text{peak FLOPs} \times \text{GPUs})$. Target >40%, alarm <30%
- Loss curve: smooth descent = healthy. Spike = gradient overflow; plateau = data/convergence issue
- Gradient norm: sudden spike + loss jump = instability. Use max_norm=1.0 clipping
- Experiment tracking: W&B recommended for cross-run comparison + hyperparameter sweeps

- **NCCL Debug Logging**

- NCCL_DEBUG=INFO: algorithm selection (ring/tree/collnet), chunk sizes, protocol
- NCCL_DEBUG=WARN: warnings only — use in production
- NCCL_ALGO / NCCL_PROTO: override to test different algorithms
- Ring = optimal for large-message AllReduce on InfiniBand
- Correlate NCCL logs with DCGM NVLink/IB metrics to distinguish SW config vs HW fault

Slurm Job Monitoring Commands

```
# View running and pending jobs; check 'REASON' column
squeue -u <user>
# PD reasons: Priority, Resources, launch_failed

# Post-run accounting for cost attribution
sacct -j <jobid> --format=JobID,Elapsed,CPUTime,MaxRSS,State

# Detailed resource allocation, node list
scontrol show job <jobid>

# Hyperparameter sweeps via job arrays
#SBATCH --array=1-100
# sacct aggregates across all array tasks automatically
```

- squeue REASON codes to know: Resources (waiting for GPUs), Priority (lower-priority job), launch_failed (check stderr logs)
- sacct for billing: $\text{Elapsed} \times \text{GPU count} \times \$/\text{GPU-hr}$ = job cost for attribution to teams/experiments
- **Job arrays are the idiomatic Slurm pattern for hyperparameter sweeps — one sbatch submission, N parallel tasks**

Troubleshooting & Common Pitfalls

Common Pitfalls: Programming & Stability

RNG Race Conditions

Forget to seed per-rank → all GPUs shuffle identically.
Fix: `torch.manual_seed(42 + dist.get_rank())`

Batch Size Scaling Effects

2× GPUs = 2× effective batch size → learning dynamics change.
Fix: scale LR linearly with batch size + linear warmup period.

DDP Checkpoint Key Mismatch

Saving `model.state_dict()` instead of `model.module.state_dict()` → keys have 'module.' prefix → load fails.
Fix: always use `.module` to unwrap DDP.

Numeric Instability (Multi-GPU)

Mixed precision across many GPUs can amplify rounding errors.
Fix: use BF16 (wider range), apply gradient clipping `max_norm=1.0`, monitor grad norm in W&B.

Common Pitfalls: Infrastructure & Scaling

Deadlock — Misconfigured Process Groups

Wrong rank/world_size → processes wait forever for rendezvous.

Fix: double-check global_rank calculation; add NCCL timeout; use NCCL_DEBUG=INFO.

Pipeline Bubble Waste

PP stages idle >12% of time.

Fix: increase micro-batches per stage (target ≥ 8). Check: $\text{bubble} \approx (\text{stages}-1)/(\text{stages}-1+\mu\text{batches})$.

Communication-Bound Scaling

Adding GPUs doesn't improve speed.

Fix: verify IB is active (network_tier: best); run all_reduce_perf; check MFU. Target: ring allreduce on IB.

Gradient Sync Failures

Sudden loss spike / NaN → possible network interruption or HW fault.

Fix: check DCGM for XID errors; check NCCL logs; verify ECC counters. Transient = retry; persistent = drain node.

Thank You

Questions & Discussion